

Aufgabe 12

Aufgabe 12.1

Erklärung:

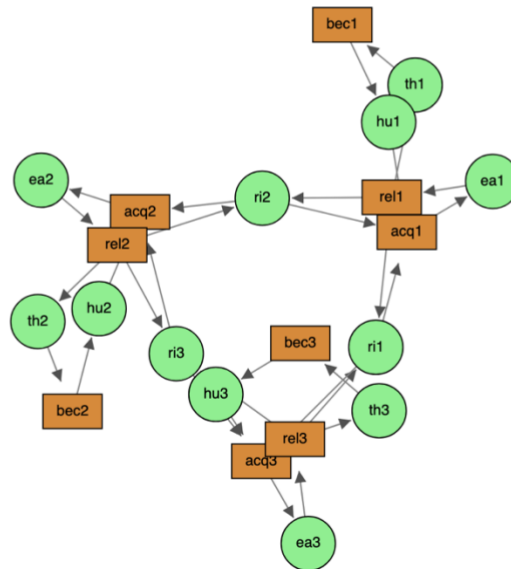
Aufgabe 12.1 baut das Beispiel *Force Directed Graph* aus `gampleman/elm-visualization` so um, dass es das Petri-Netz des Philosophen-Dilemmas zeichnet. Ein Petri-Netz hat zwei Knotentypen – *Stellen* (Kreise: th, hu, ea, ri) und *Transitionen* (Rechtecke: bec, acq, rel) – die sich in einer einzigen Graph-Datenstruktur unterbringen lassen, indem ein eigener Label-Typ `NodeData = { name, shape }` mit `NodeShape = Place | Transition` eingeführt wird; beim Rendern entscheidet das `shape`-Feld pro Knoten, ob ein `circle` oder ein `rect` ausgegeben wird. Der Graph selbst entsteht über `Graph.fromNodeLabelsAndEdgePairs` aus `elm-`

`community/graph`: `philoNodes` erzeugt für jeden der drei Philosophen $i \in \{1,2,3\}$ die sieben Knoten mit deterministischem Index ($\text{base } i = (i-1)*7$), und `philoEdges` listet pro Philosoph zehn gerichtete Kanten auf, die den klassischen Zyklus $\text{th} \rightarrow \text{bec} \rightarrow \text{hu} \rightarrow \text{acq} \rightarrow \text{ea} \rightarrow \text{rel} \rightarrow \text{th}$ modellieren und über die geteilten Gabeln $\text{ri}_i / \text{ri}_{(i+1)}$ die Nachbarn koppeln ($\text{next } i = \text{modBy } 3 \ i + 1$). Für das Layout läuft eine `Force.simulation` mit drei Kräften: `Force.customLinks 1` mit fester `distance = 70` hält verbundene Knoten auf Federabstand, `Force.center` zieht den Schwerpunkt in die SVG-Mitte und – wie in der Aufgabe gefordert – `Force.customManyBody 0.9 [(id, -90), ...]` ersetzt das `Standard-manyBody`; der negative Wert `-90` (statt der Default `-30`) erzeugt eine deutlich stärkere Abstoßung, damit die größeren Rechtecke und Kreise sich nicht überlappen. Pro Animationsframe (`Browser.Events.onAnimationFrame`) ruft `update` ein `Force.tick` auf und schreibt die neuen (x, y) zurück in den Graphen; sobald `Force.isCompleted` wahr ist, wird das Abo abbestellt, um CPU zu sparen. Drag-and-Drop ist über `mpizenberg/elm-pointer-events` realisiert: beim `mousedown` auf einem Knoten wird die `Nodeid` im `Drag-Record` gemerkt, jedes `mousemove` aktualisiert dessen Position und reizt die Simulation mit `Force.reheat neu`. Die Pfeilspitzen sind ein wiederverwendbarer `SVG-<marker id="arrow">`, der per `markerEnd="url(#arrow)"` an jede Kante gehängt wird; damit der Pfeil nicht im Knoten verschwindet, verschiebt `endPoint` den Endpunkt entlang der Verbindungslinie um den Knotenradius zurück – exakt bei Kreisen, näherungsweise (mit der halben Rechteckbreite) bei Rechtecken, wie in der Aufgabenstellung ausdrücklich gestattet.

Screenshot:

Übung 12.1 – Philosophen-Dilemma (Petri-Netz)

Knoten lassen sich mit der Maus ziehen.



Link: <https://ellie-app.com/yTH37T29fKca1>

Aufgabe 12.2

Erklärung:

Aufgabe 12.2 baut direkt auf dem Beispiel ForceDirectedGraph mit dem *Les-Misérables*-Datensatz auf. Statt eine feste Kraftkonfiguration zu verwenden, wird die Funktion `buildSimulation : Scenario -> Graph Entity () -> State NodeId` eingeführt, die je nach gewähltem Szenario eine andere Liste von Kräften an `Force.simulation` übergibt. Über ein `<select>`-Menü oben in der Ansicht kann zwischen acht vordefinierten Szenarien umgeschaltet werden; bei jedem Wechsel wird der Graph mit `Graph.mapContexts initializeNode miserablesGraph` frisch initialisiert und die Simulation komplett neu aufgebaut, damit man jedes Szenario unverfälscht von den vorherigen Knotenpositionen beobachten kann. Die drei Standardkräfte des Beispiels wirken jeweils unterschiedlich auf das Layout. `Force.links` modelliert die Kanten als Federn, die verbundene Knoten auf

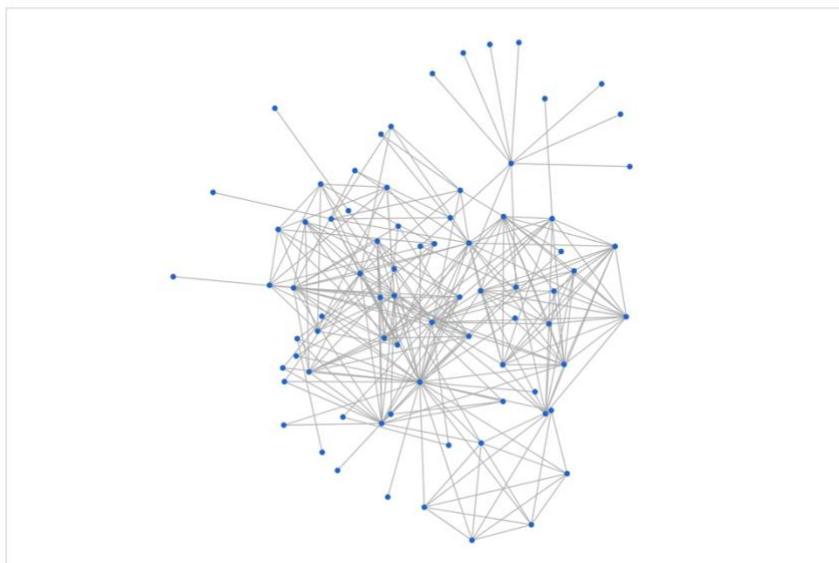
einer Soll-Distanz halten und so die Topologie des Graphen sichtbar machen. Force.manyBody weist jedem Knoten eine negative „elektrische Ladung“ zu, sodass sich alle Knoten gegenseitig abstoßen und nicht zu einem Punkt kollabieren. Force.center verschiebt schließlich den Schwerpunkt der gesamten Simulation in die SVG-Mitte und verankert das Bild visuell. Im *Default*-Szenario halten sich diese drei Kräfte die Waage und erzeugen die typische, lesbare „Wolke“ mit erkennbaren Cliquen. Die übrigen Szenarien isolieren oder verstärken jeweils eine dieser Kräfte, um ihre Wirkung erfahrbar zu machen. *Ohne center* driftet der Graph aus der Mitte, was zeigt, dass center nur die Verankerung, nicht aber die Struktur erzeugt. *Ohne manyBody* fehlt der Gegenspieler zu den Federn, alle Knoten kollabieren zu dichten Klumpen; *ohne Links* bleibt umgekehrt nur eine gleichmäßige Gas-Verteilung, in der die Topologie verloren geht. Über Force.customManyBody lässt sich die Ladung gezielt steuern – mit einer sehr starken Abstoßung (-300) quillt der Graph bis an die Ränder und Cliquen werden klar getrennt, mit einer positiven Ladung ($+30$) kehrt sich das Vorzeichen um und alle Knoten ziehen sich zu einem einzigen Punkt zusammen. Analog erlaubt Force.customLinks, die Federdistanz frei zu setzen: kurze Distanzen (15) erzeugen ein sehr dichtes, gestauchtes Bild, lange Distanzen (120) ein weit aufgezoogenes Layout, bei dem die Übersicht steigt, kleine Untergruppen aber verschwimmen. Die zugehörigen Beobachtungen sind direkt in der Ansicht als `<ul>` unter dem SVG aufgelistet, sodass beim Umschalten des Szenarios die visuelle Veränderung und die textuelle Beschreibung unmittelbar abgeglichen werden können.

Screenshot:

Übung 12.2 – Experimente mit Kräften

Szenario:

Aktuell: customLinks: lange Distanz (120)



Beobachtungen

- **Default:** Drei Kräfte halten sich die Waage – Links ziehen verbundene Knoten zusammen, manyBody stößt alle voneinander ab, center hält den Schwerpunkt in der SVG-Mitte. Ergebnis: kompakte, lesbare Wolke um die Mitte.
- **Ohne center:** Der Schwerpunkt ist nicht mehr fixiert. Der Graph driftet (je nach Anfangsbedingung) langsam aus der Mitte heraus, behält aber seine innere Form. center erzwingt also nur die Verankerung, nicht die Struktur.
- **Ohne manyBody:** Ohne die abstoßende Ladung fallen alle verbundenen Knoten zu kleinen Klumpen zusammen, weil die Link-Federn nichts gegenhalten. Isolierte Komponenten kleben aufeinander.
- **Ohne Links:** Die Kantenstruktur spielt keine Rolle mehr; manyBody verteilt die Knoten gleichmäßig wie ein elektronisches Gas, center hält sie zusammen. Man sieht zwar Linien, aber die Topologie ist verloren.
- **Starke Abstoßung (-300):** Knoten drücken sich weit auseinander, der Graph quillt bis an die Ränder. Strukturen wie Cliques werden klarer voneinander getrennt, aber die Federn der Links werden stark gedehnt.
- **Anziehung (+30):** Positives manyBody invertiert die Ladung: alle Knoten ziehen sich gegenseitig an und kollabieren zu einem dichten Punkt – die Visualisierung ist praktisch unbrauchbar, schön zeigt sich aber, wie wichtig die Abstoßung ist.
- **Kurze Links (15):** Federn sind sehr kurz; benachbarte Knoten klemmen aneinander. Der Graph wird kleiner und dichter, Cluster wirken kompakter.
- **Lange Links (120):** Federn sind weich und lang. Der Graph dehnt sich aus, die Übersicht ist besser, aber kleine Untergruppen verschwimmen, weil die Abstoßung relativ schwächer wirkt.

Link: <https://ellie-app.com/yTH7VpmSqFH1>